

CS545 - Cloud Native Platform Engineering

Anthi Lyra

Lab 2

04/19/2024

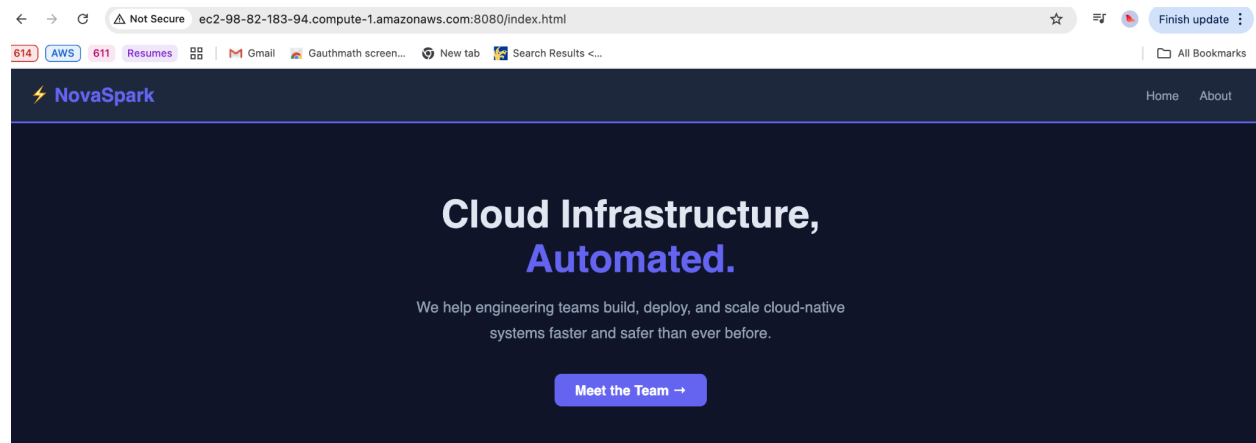
D1.

```
Do you want to perform this update? yes
Updating (dev):
  Type                Name                Status
+ pulumi:pulumi:Stack ec2-lab-dev         created (23s)
+   └─ aws:ec2:SecurityGroup ssh-web-access      created (3s)
+   └─ aws:ec2:Instance    lab-instance        created (13s)

Outputs:
publicDns : "ec2-98-82-183-94.compute-1.amazonaws.com"
publicIp  : "98.82.183.94"
sshCommand: "ssh -i ~/.ssh/labsuser.pem ec2-user@ec2-98-82-183-94.compute-1.amazonaws.com"
websiteUrl: "http://ec2-98-82-183-94.compute-1.amazonaws.com:8080"

Resources:
+ 3 created
```

D2.



D3.

```
[ec2-user@ip-172-31-36-181 ~]$ aws configure list
NAME      : VALUE                                : TYPE      : LOCATION
profile   : <not set>                               : None      : None
access_key : *****C4JN                            : iam-role   :
secret_key : *****a9h                             : iam-role   :
region    : us-east-1                              : imds       :
[ec2-user@ip-172-31-36-181 ~]$
```

D4.

```

http://169.254.169.254/latest/meta-data/iam/security-credentials/$ROLE_NAME | python3 -m json.tool
IAM Role attached to this instance: LabRole
{
  "Code": "Success",
  "LastUpdated": "2026-04-16T01:50:09Z",
  "Type": "AWS-HMAC",
  "AccessKeyId": "ASIATCKARDC43SNLC4JN",
  "SecretAccessKey": "YM7snKkVqXZ8s0CRh1bM/++wFVjM7SbgGMC/+a9h",
  "Token": "IQoJb3JpZ2luX2VjE0r////////wEaCXVzLWVhc3QtMSJHMEUCIQC37rwV9ZRgsHor6PSGDzL6TVazH+9cuSH6JWaGfbh0ugIg0xT22Y
DUiDJU7fgqxAARD8kUuoYqZBTYAC/5t3CdYwyt802gZDGG28ScW4etjUB984hhuXbbtc6J9IhmUd40sLP0mBMK0Uc8kaiod2sc+R+M5jKzctlkEf8w2YT4T
0gfL+t8R/iHXtEFJFGxtaQaU00T0L93+AvCt+n0Ueb8GBUqwa+AtyRwWDMe/0LY+MfZBmnLAQfQMWUMfIeqCB0EzkNnr+i2RDYsv9yMDiAEUwAsbP81X16dP
NALoSj+Z7yEwde0J0H95Bf90F192LW40iJY4qJUKSYJRjQxgVLUFU5KGQIB0f3UU0FpEuT0Zj869AKR76syqmpmDMWbPTltYlUeBNaxPKqvqtxlkQ/WnsvuS
WjbmPJlrrhhN05F/sV4iYzbCTjLshyDb9c0sS7ZD2zLGqn0Sv+cLLLR5fqCLWp5Q/mvHkQ2X8YofL5t+UGGLN/v3U1v7W12hd7fRt5G0h0TcQKkcvSW5rVYE
e0w0t0vCqL95ZohS04Dx7rtE1BvrUuOWi/nf7foJDsB9bE3E87X4yXkfQeJc0LHUlpnTAQf5INLn7VUEoT1xIv3Fyn4n81KI9hcRIL6fnjX4srSnlfzVUuv/X
KIB0tJPiLNQaaX2ApJUZwb0mt/Rb0wzAvXqx/gEjWybzXjhMgzSLd60z1e6Ziaec9CC6NT+deqlRqDNtqEjYG060SCAxEqxaYl5q311wRozjNyw1UvKb4hw
"Expiration": "2026-04-16T08:25:27Z"
}
[ec2-user@ip-172-31-36-181 ~]$

```

D5.

```

</html>
[ec2-user@ip-172-31-36-181 website]$ aws s3 ls s3://$BUCKET_NAME/
2026-04-16 02:08:24      1916 about.html
2026-04-16 02:08:15      2101 index.html
[ec2-user@ip-172-31-36-181 website]$

```

D6.

```

< Server: BaseHTTP/0.6 Python/3.9.25
< Date: Thu, 16 Apr 2026 02:23:10 GMT
< Content-Type: text/html; charset=utf-8
< X-Served-From: S3
<
  0      0      0      0      0      0      0      0  --:--:-- --:--:-- --:--:--      0{ [2101 bytes data]
100 2101      0 2101      0      0 8527      0  --:--:-- --:--:-- --:--:--      8506
* Closing connection
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>NovaSpark Technologies</title>
  <style>

```

D7.1

```

Connection to ec2-98-82-183-94.compute-1.amazonaws.com closed.
(base) anthilyra@Anthis-MacBook-Air lab-p1 % pulumi destroy
Previewing destroy (dev):
  Type                               Name                Plan
  -   ┌─ pulumi:pulumi:Stack          ec2-lab-dev         delete
  -   │   ┌─ pulumi:pulumi:Stack      ec2-lab-dev         delete
  -   │   │   ┌─ aws:ec2:SecurityGroup ssh-web-access      delete
  -   │   │   └─ aws:ec2:Instance     lab-instance        delete

Resources:
  - 3 to delete

Do you want to perform this destroy? yes
Destroying (dev):
  Type                               Name                Status
  -   ┌─ pulumi:pulumi:Stack          ec2-lab-dev         deleted (0.00s)
  -   │   ┌─ pulumi:pulumi:Stack      ec2-lab-dev         deleted (30s)
  -   │   │   ┌─ aws:ec2:Instance     lab-instance        deleted (30s)
  -   │   │   └─ aws:ec2:SecurityGroup ssh-web-access      deleted (0.83s)

Resources:
  - 3 deleted

Duration: 37s

The resources in the stack have been deleted, but the history and configuration associated with the stack are still maintained.
If you want to remove the stack completely, run `pulumi stack rm dev`.
(base) anthilyra@Anthis-MacBook-Air lab-p1 %

```

D7.2

```
(base) anthilyra@Anthi-MacBook-Air lab-p2 % aws s3 ls s3://$(pulumi stack output bucketName)
(base) anthilyra@Anthi-MacBook-Air lab-p2 % pulumi destroy
Previewing destroy (dev):
  Type                                         Name                                         Plan
- pulumi:pulumi:Stack                         ec2-lab-2g-dev                             delete
-   └─ pulumi:pulumi:Stack                    ec2-lab-2g-dev                             delete
-     └─ aws:s3:BucketV2                       novaspark-website                         delete
-       └─ aws:ec2:SecurityGroup               web-ssh-access                           delete
-         └─ aws:s3:BucketPublicAccessBlock    novaspark-website-block-public           delete
-           └─ aws:ec2:Instance                lab2-part2-server                        delete
-             └─ aws:iam:InstanceProfile       ec2-instance-profile                     delete

Resources:
- 6 to delete

Do you want to perform this destroy? yes
Destroying (dev):
  Type                                         Name                                         Status
- pulumi:pulumi:Stack                         ec2-lab-2g-dev                             deleted (0.00s)
-   └─ pulumi:pulumi:Stack                    ec2-lab-2g-dev                             deleted (1s)
-     └─ aws:s3:BucketPublicAccessBlock       novaspark-website-block-public             deleted (20s)
-       └─ aws:ec2:Instance                   lab2-part2-server                         deleted (0.50s)
-         └─ aws:iam:InstanceProfile           ec2-instance-profile                     deleted (0.39s)
-           └─ aws:s3:BucketV2                novaspark-website                         deleted (0.87s)
-             └─ aws:ec2:SecurityGroup         web-ssh-access                           deleted (0.87s)

Resources:
- 6 deleted

Duration: 25s

The resources in the stack have been deleted, but the history and configuration associated with the stack are still maintained.
If you want to remove the stack completely, run `pulumi stack rm dev`.
(base) anthilyra@Anthi-MacBook-Air lab-p2 %
```

D8.

When the EC2 instance starts, it has an instance profile attached with an IAM role that defines what it was allowed to do, like reading from in our lab an S3 bucket. The instance then contacted the Instance Metadata Service (IMDS) at 169.254.169.254 to get temporary credentials, which had an AccessKeyId starting with “ASIA” and an expiration time, which means these credentials only last a few hours (D4 screenshot). The boto3 library automatically uses these credentials, so there are no access keys stored in the code. When the web server calls `get_object()`, IAM checks if the role has permission to access the S3 bucket, and if it does, S3 returns the file, otherwise it would return a AccessDenied 403 error. The X-Served-From: S3 header proved that everything worked correctly because it is only added after a successful response from S3, meaning the chain worked.

D9.

By default, EC2 instances have no credentials and can not access any AWS services. As mentioned in the lab, when we ran `aws s3 ls` and got Access Denied error, and `aws configure list` showed all values as None. This is called default-deny, which means nothing is allowed unless we explicitly give permission. The problem with not having this is that if any EC2 instance could read any S3 bucket automatically, one hacked instance could access our entire data lake. The IAM role pattern we used fixes this because each instance only gets access to the specific bucket it needs, so even if something gets compromised the damage stays limited.

D10.

Pattern	Failure mode	Risk severity
Environment Variables	Anyone with shell access can just run <code>echo\$AWS_ACCESS_KEY_ID</code> and see the credentials.	Medium/High
<code>~/.aws/credentials</code> on disk	If files get leaked, shared or we accidentally pushed to Github, anyone could use those credentials	High
Hardcoded in application code	This is the worst because credentials can be exposed in source code, Github, or shared code making them easy to steal	Very High
IAM role via IMDS	Credentials are temporary, expire after a few hours and are not stored anywhere permanently.	Low

D11.

Dimension	Part 1	Part 2
Storage	Files are stored directly on the EC2 instance and gone if instance reboots	Files are stored in S3, survive any instance event
Security/ Credentials	No IAM roles, aws configure list showed None	IAM roles attached(temporary ASIA and credentials from IMDS)
Configuration	Create HTML files, start <code>python3 -m http.server 8080</code>	Upload files with <code>aws s3 cp</code> , install boto3, run <code>s3_webserver.py</code>
Cleanup	Just <code>pulumi destroy</code>	Had to empty S3 bucket first with <code>aws s3 rm</code> then <code>pulumi destroy</code>
Setup	Everything is done manually through SSH after launching the instance	Still need to SSH in and manually install boto3 and run the server

Part 3's `user_data` basically eliminates everything we had to do manually in Parts 1 and 2, such as SSH, creating the HTML files, installing boto3, uploading files to S3, and starting the server. Instead of doing all these steps by hand, the instance runs a startup script automatically when it boots. This makes the whole setup much more consistent because every instance follows the same exact process. In a production environment with 50+ instances, this is important because manually SSHing into each one would take too long and would lead to mistakes. Even one missed step could break a single instance and make it behave differently from the others. With `user_data`, everything becomes automated and repeatable, so scaling the system is much easier and more reliable.

Content Paragraph

Infrastructure as Code (IaC) is the idea of defining infrastructure using code instead of setting things up manually, so everything can be created in a repeatable way (Fowler). Fowler explains that IaC makes infrastructure more consistent because everything is written down and version controlled, but it can also cause issues when people assume the code guarantees everything will work without understanding what is actually happening in the system. In Lab 2 Part 2 Step 3, I SSHed into the instance and ran `aws s3 ls` and got Access Denied, even though Pulumi had already deployed everything correctly. This showed that even if the code is correct, the system is not always ready right away because the IAM role has not fully updated yet. Then in Part 2 Step 5, I ran `aws configure list` and it finally showed `iam-role` in the Type column, meaning the instance was now automatically getting temporary credentials from IMDS. This makes Fowlers idea a bit more complicated because IaC makes systems feel predictable, but in reality there are still delays and AWS background processes that are not visible in the code. The lab showed me that IaC improves consistency, but you still need to understand what is actually happening for it to fully make sense.

Extra Credit

EC1.

```
Outputs:
bucketName      : "novaspark-website-1295c18"
checkBootstrap  : "ssh -i ~/.ssh/labsuser.pem ec2-user@ec2-3-87-17-168.compute-1.amazonaws.com 'sudo journalctl -u novaspark-web -n 30'"
publicDns       : "ec2-3-87-17-168.compute-1.amazonaws.com"
publicIp        : "3.87.17.168"
sshCommand      : "ssh -i ~/.ssh/labsuser.pem ec2-user@ec2-3-87-17-168.compute-1.amazonaws.com"
websiteUrl      : "http://ec2-3-87-17-168.compute-1.amazonaws.com:8080"

Resources:
+ 8 created

Duration: 30s
```

EC2.

The screenshot shows a web browser displaying the NovaSpark website. The browser's address bar shows the URL `ec2-3-87-17-168.compute-1.amazonaws.com:8080/index.html`. The website has a dark blue theme with a white NovaSpark logo in the top left. A navigation bar at the top right contains links for "Home" and "About". A central banner features the text "NOW HIRING CLOUD ENGINEERS" in a blue pill-shaped button, followed by the main heading "Cloud Infrastructure, Automated." in large white and blue font. Below this, a subheading states: "We help engineering teams build, deploy, and scale cloud-native systems — faster, safer, and with dramatically less toil." Two buttons, "Meet the Team" and "See How We Work", are positioned below the text. At the bottom, a section highlights four key metrics: "99.9% UPTIME SLA", "< 5 min DEPLOY TIME", "0 HARD-CODED CREDENTIALS", and "100% INFRASTRUCTURE AS CODE".

EC3.

```
• (base) anthilyra@Anthis-MacBook-Air lab-p3 % pulumi stack output
Current stack outputs (6):
OUTPUT      VALUE
bucketName  novaspark-website-1295c18
checkBootstrap ssh -i ~/.ssh/labsuser.pem ec2-user@ec2-3-87-17-168.compute-1.amazonaws.com 'sudo journalctl -u novaspark-web -n 30'
publicDns    ec2-3-87-17-168.compute-1.amazonaws.com
publicIp     3.87.17.168
sshCommand   ssh -i ~/.ssh/labsuser.pem ec2-user@ec2-3-87-17-168.compute-1.amazonaws.com
websiteUrl   http://ec2-3-87-17-168.compute-1.amazonaws.com:8080
○ (base) anthilyra@Anthis-MacBook-Air lab-p3 %
```

EC4.

```
The authenticity of host 'ec2-3-87-17-168.compute-1.amazonaws.com (3.87.17.168)' can't be established.  
ED25519 key fingerprint is SHA256:FtqaVoROTikQAamFnFqxhFNvFW8fg07DDTp07wZS9rk.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'ec2-3-87-17-168.compute-1.amazonaws.com' (ED25519) to the list of known hosts.  
Apr 16 02:53:35 ip-172-31-38-104.ec2.internal systemd[1]: Started novaspark-web.service - NovaSpark S3 Web Server.  
Apr 16 02:53:35 ip-172-31-38-104.ec2.internal python3[2106]: /usr/local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting  
April 29, 2026. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/dev  
eloper/python-support-policy-updates-for-aws-sdks-and-tools/  
Apr 16 02:53:35 ip-172-31-38-104.ec2.internal python3[2106]: warnings.warn(warning, PythonDeprecationWarning)  
(base) anthilluz@Anthic-MacBook-Air lab-c3 %
```

When I ran the `checkBootstrap` command, `journalctl` showed that the `novaspark-web.service` started successfully with no errors, and the website was live after about 90 seconds. The only issue I saw was a `PythonDeprecationWarning` saying that `boto3` will drop support for Python 3.9 in April 2026, which means this setup could break in the future without any changes. One thing I would change is updating the bootstrap script to install Python 3.10 or higher instead of using the default Python 3.9. This is important because if a new instance is created after April 2026, the web server might fail during startup without showing an error.